AYAAM – Project Documentation

1. Introduction :

AYAAM is a Single Page Application (SPA) E-commerce platform designed to serve three main roles: Customer, Seller, and Admin.
It provides a smooth browsing and shopping experience using JavaScript modules, hash-based routing, and lazy-loaded pages.
The project is built with plain JavaScript, Bootstrap, CSS, and FontAwesome without external frameworks or APIs.
All data is handled using LocalStorage and SessionStorage for persistence.

GitHub Repository : Githup

Live Demo : AYAAM.com

2. Features :

2.1 Customer

Sign up and login

Browse products by category, brand, filters

Add products to cart

Checkout flow with order confirmation

View and manage profile information

2.2 Seller

Add, edit, and delete products

Manage product stock, discounts, and offers

View sales dashboard with performance insights

2.3 Admin

Manage users (customers & sellers)

Manage all products

Monitor system reports

Role-based access control

3. Technical Architecture

3.1 SPA and Routing

- Single Page Application using Native Javascript
- Hash-based routing (#/home, #/catalog, #/product)
- Router module handles navigation and lazy loads the required page modules.
- Role-based protection:
    - Customer-only routes (checkout, profile, cart)
    - Seller-only routes (seller dashboard)
    - Admin-only routes (admin dashboard)
- Fallback to /404 page for unknown routes.

3.2 Data Management

- **LocalStorage**: Products, Users, Orders (seeded on first load)
- **SessionStorage**: User sessions, redirects, and temporary states
- No backend / API integration (mock data only)

3.3 Shared Components

- components/ → reusable UI building blocks
    - Cart
    - Landing
    - Dashboard elements
    - Layout
    - UI
- scripts/utils/ → helper functions
    - Loader utility
    - Navigation
    - Storage abstraction
    - Data seeding

3.4 Security & Role Management :

role-based access for Admin, Seller, Customer .

3.5 Technology Stack

Plain JS, Bootstrap, CSS, Font Awesome, LocalStorage/SessionStorage .

4. Project Structure :

E-COMMERCE-ITI

│── assets/ # CSS, images, JS assets

│── components/ # Reusable components (auth, cart, profile, etc.)

│── data/ # Local JSON/data & seeders

│── pages/ # SPA pages (Home, Catalog, Product, Cart, Admin, Seller, etc.)

│── scripts/ # Core scripts (router, utils, cartScripts)

│── theme-files/ # Extra theme/config files

│── index.html # Entry point

│── app.js # App initialization & routing

│── **README.md**

5. Data Model :

The project uses object-oriented modules (User, Product, Order) to structure and enforce data consistency. These classes wrap the raw JSON objects and add validation, formatting, and serialization (toJSON()) for safe storage in LocalStorage / SessionStorage.

4.1 User Model (User Class) :

Represents system users with role-based access control

```
export default class User {
  #id; #name; #email; #password; #role;
  #phone; #status; #joinDate;

  constructor(
    _id, _name, _email, _password,
    _role = 'user', _phone = '',
    _status = 'active', _joinDate = null
  ) { ... }

  // Getters & setters for all fields
  // Role = "master" | "admin" | "seller" | "user"
  // Status = "active" | "inactive"

  toJSON() {
    return {
      id: this.#id,
      name: this.#name,
      email: this.#email,
      password: this.#password,
      role: this.#role,
      phone: this.#phone,
      status: this.#status,
      joinDate: this.#joinDate
    };
  }
}
```

Notes

- Enforces roles (master, admin, seller, user).
- Defaults to "user" role.
- toJSON() makes the object serializable for LocalStorage.
- Plain-text passwords only for prototype/demo purposes.

4.2 Product Model (Product Class) :

Represents catalog products, including categories, variants, and inventory.

```
export default class Product {
  #id; #name; #description; #category; #subcategory;
  #price; #sale; #offers; #stock;
  #brand; #material; #sellerId; #status;

  constructor(
    _id, _name, _description,
    _category, _subcategory,
    _price = 0, _sale = 0,
    _offers = [], _stock = [],
    _brand = '', _material = '',
    _sellerId = '', _status = 'pending'
  ) { ... }

  // Getters/setters with validation
  // - Name auto-formatted (capitalized words)
  // - Price must be number
  // - Sale must be 0–1 fraction
  // - Stock must be array
  // - Status defaults to "pending"

  isApproved() { return this.#status === 'approved'; }

  toJSON() { ... }
}
```

Notes

- Enforces strict validation on setters (e.g., Sale between 0 and 1).
- Normalizes capitalization for Name, Category, Subcategory, and Brand.
- Stock structure:

```
stock: [
  {
    color: "Black",
    images: ["img1.png", "img2.png"],
    sizes: [
      { name: "M", qty: 2 },
      { name: "L", qty: 3 }
    ]
  }
]
```

- isApproved() is used to filter products in the catalog.

4.3 Order Model (Order Class)

Encapsulates customer orders with private fields and auto-calculated totals.

```
export default class Order {
  #id; #customerId; #items; #total;
  #address; #payment; #status;

  constructor(
    _id, _customerId,
    _items = [], // [{ productId, color, size, qty, price }]
    _address, _payment = "Cash",
    _status = "processing"
  ) { ... }

  // Auto-calculates total when items are set
  #calculateTotal() { ... }

  toJSON() {
    return {
      id: this.#id,
      customerId: this.#customerId,
      items: this.#items,
      total: this.#total,
      address: this.#address,
      payment: this.#payment,
      status: this.#status
    };
  }
}
```

Notes

- Total is always consistent, recalculated automatically.
- Payment defaults to "Cash".
- Status workflow: "processing" → "shipped" → "completed" | "canceled".

4.4 Relationships

- **User → Product**
  - Sellers (role: seller) own multiple products via Product.SellerId.
- **User → Order**
  - Customers (role: user) create orders via Order.CustomerId.
- **Product → Order**
  - Orders reference products by productId inside Order.Items[].

6. Pages & Views (SPA Routes) :

AYAAM is built as a Single Page Application (SPA) using a custom Router that listens to hashchange and DOMContentLoaded events.

Each route corresponds to a page module dynamically imported (lazy-loaded) only when needed. This optimizes performance and reduces initial load size.

5.1 Public Routes

Route	View Module	Access	Description
/home	HomePage.js	Public	Landing page with featured products and categories.
/about	AboutPage.js	Public	Information about the project/brand.
/catalog	CatalogPage.js	Public	Displays product list with filters (brand, category, price, etc.).
/product	Product.js	Public	Single product details page with images, sizes, colors, and add-to-cart.
/info	SupportPage.js	Public	Customer support, FAQs, or static info page.
/404	NotFound.js	Public	Fallback page for invalid routes.

5.2 Authentication Routes

Route	View Module	Access	Description
/login	LoginPage.js	Public	User login form, authenticates and stores session in SessionStorage.
/signup	SignupPage.js	Public	Registration form for new customers/sellers.

5.3 Customer Routes

Route	View Module	Access	Description
/cart	CartPage.js	Customer only	Shopping cart, manages items before checkout.
/checkout	CheckOutPage.js	Customer only	Checkout flow with address, payment, and confirmation.
/profile	Profile.js	Customer only	User profile: personal info, past orders, settings.

5.4 Seller Routes

Route	View Module	Access	Description
/seller	SellerDashboard.js	Seller only	Dashboard for sellers: add/edit/delete products, view sales.

5.5 Admin Routes

Route	View Module	Access	Description
/admin	AdminDashboard.js	Admin/Master	Manage users, approve/reject products, oversee reports.

5.6 Routing Logic

- Routes are matched via **longest prefix match** in Router.resolve().
- Restricted routes check the current user role before rendering.
  - If unauthorized → redirect to /404.
  - If unauthenticated → redirect to /login.
- Dynamic imports ensure **lazy-loading**:

```
"/catalog": { loader: () => import("./pages/CatalogPage.js") }
```

7. User Flows

- **Customer flow**: Signup → Browse → Add to cart → Checkout → Profile
- **Seller flow**: Login → Add products → Manage stock/offers → View dashboard
- **Admin flow**: Login → Manage users → Manage products → Reports

8. Team Contributions

The development of **AYAAM** was a collaborative effort by a 5-member team. Each member was responsible for specific modules and features, ensuring clear ownership and efficiency.

Team Member	Contributions
Yasser	- Implemented SPA project Archeticture. - Handled protected routing and fallbacks. -Implemented **Authentication & Authorization** system. - Built **Login** & **Signup** flows. - Handled **Role-Based Access Control (RBAC)** across routes. - Managed **user session state** using LocalStorage/SessionStorage. - Developed **Home Page** UI and logic.
Ahmed beltagy	- Developed **Product Catalog** (Product List with filters, search). - Implemented **Product Card component** and reusable catalog UI. - Designed and coded **Filter System** (category, brand, size, color, price, discount, offers). - Built **Single Product Page** with images, details, and add-to-cart.
Azza	- Implemented **Checkout Page** (address, payment, confirmation). - Developed **Order saving mechanism**. - Added **Order History** view for customers. -Added Profile Page for users
Mariam	- Developed **Seller Dashboard**. - Features: **Add / Edit / Delete products**, manage inventory (stock, price, discount). - Built **Incoming Orders page** for sellers. - Added **Sales History tracking**.
Ahmed Osama	- Implemented **Admin Dashboard**. - Built full **User Management (CRUD)** system. - Developed **Product Review & Approval workflow**. - Oversaw **Reports & Control Panel** for admins/master role.

9. Conclusion

AYAAM demonstrates a **modular SPA architecture** for an e-commerce platform using only **Vanilla JS** with Bootstrap styling.
It provides **role-based features** for customers, sellers, and admins, showcasing real-world flows in a simplified educational project.